
Monte Carlo Simulation and Statistical Modeling

*Analysis of Complex Probability Distributions,
CLT Convergence Verification,
and European Option Pricing*

Abdellah Kahlaoui

Master of Applied Mathematics
FST Settat, Morocco

kahlaouiabdellah6@gmail.com
github.com/AbdellahKah

November 2025 – January 2026

Source code: github.com/AbdellahKah/monte-carlo-statistical-modeling

Abstract

This report presents a comprehensive Monte Carlo simulation study of four canonical probability distributions: Normal, Log-Normal, Exponential, and Poisson. We develop a vectorized simulation engine in Python capable of generating 10^6 independent paths per distribution in under one second. Empirical moments are validated against closed-form theoretical values, and goodness-of-fit is assessed via Kolmogorov-Smirnov and chi-squared tests. We experimentally confirm the convergence rate predicted by the Central Limit Theorem, measuring log-log slopes between -0.487 and -0.509 across all distributions. Finally, we apply the Monte Carlo method to the pricing of a European call option under Black-Scholes assumptions, achieving a price estimate of 10.4284 against the analytical value of 10.4506 , a relative error below 0.3% at $n = 10^6$ paths.

Contents

1	Introduction	2
2	Theoretical Background	2
2.1	Probability Distributions	2
2.2	The Central Limit Theorem	2
2.3	Goodness-of-Fit Testing	3
3	Implementation	3
3.1	Simulation Engine	3
3.2	Convergence Analyser	3
4	Results — Distribution Analysis	3
4.1	Empirical vs Theoretical Moments	3
4.2	Goodness-of-Fit Tests	4
5	Convergence Analysis	4
5.1	Experimental Convergence Rates	4
5.2	Implications for Simulation Design	4
6	Application: European Call Option Pricing	4
6.1	Black-Scholes Framework	4
6.2	Results	5
7	Conclusion	5

1 Introduction

The Monte Carlo method is a broad class of computational algorithms that rely on repeated random sampling to obtain numerical estimates of quantities that may be difficult or impossible to compute analytically. First formalised by Ulam and von Neumann in the 1940s, Monte Carlo methods are now ubiquitous across quantitative finance, statistical physics, and machine learning.

In quantitative finance, Monte Carlo simulation plays a central role in pricing path-dependent derivatives, computing Value-at-Risk and Expected Shortfall, calibrating stochastic volatility models, and simulating credit default scenarios.

The fundamental principle is straightforward: if X is a random variable with distribution F , and we wish to estimate $\mu = \mathbb{E}[g(X)]$, we generate n independent samples $X_1, \dots, X_n \sim F$ and approximate:

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n g(X_i) \xrightarrow{\text{a.s.}} \mu \quad \text{as } n \rightarrow \infty \quad (1)$$

by the Strong Law of Large Numbers. The accuracy decays as $\sigma/\sqrt{n}$ by the CLT — a rate we verify experimentally in Section 5.

2 Theoretical Background

2.1 Probability Distributions

Definition 1. $X \sim \mathcal{N}(\mu, \sigma^2)$ has PDF: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$. Moments: $\mathbb{E}[X] = \mu$, $\text{Var}(X) = \sigma^2$, skewness = 0.

Definition 2. $X \sim \text{LogNormal}(\mu, \sigma^2)$ if $\log X \sim \mathcal{N}(\mu, \sigma^2)$. Moments: $\mathbb{E}[X] = e^{\mu+\sigma^2/2}$, $\text{Var}(X) = (e^{\sigma^2} - 1)e^{2\mu+2\sigma^2}$. The log-normal is the canonical model for asset prices in the Black-Scholes framework.

Definition 3. $X \sim \text{Exp}(\lambda)$ has PDF $f(x) = \lambda e^{-\lambda x}$, $x \geq 0$. Moments: $\mathbb{E}[X] = 1/\lambda$, skewness = 2. Models inter-arrival times and default intensity in credit risk.

Definition 4. $X \sim \text{Poisson}(\lambda)$: $P(X = k) = \lambda^k e^{-\lambda}/k!$. Moments: $\mathbb{E}[X] = \text{Var}(X) = \lambda$, skewness = $1/\sqrt{\lambda}$. Models jump counts in Merton-type jump-diffusion processes.

2.2 The Central Limit Theorem

Theorem 1 (CLT). Let $X_1, X_2, \dots$ be i.i.d. with mean μ and finite variance $\sigma^2 > 0$. Then:

$$\sqrt{n} \frac{\bar{X}_n - \mu}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1) \quad \text{as } n \rightarrow \infty$$

A direct consequence is the Monte Carlo standard error:

$$SE(\hat{\mu}_n) = \frac{\sigma}{\sqrt{n}} = O(n^{-1/2}) \quad (2)$$

In log-log space: $\log SE = \log \sigma - \frac{1}{2} \log n$, so the slope is exactly -0.5 .

2.3 Goodness-of-Fit Testing

For continuous distributions we use the **Kolmogorov-Smirnov** test: $D_n = \sup_x |\hat{F}_n(x) - F_0(x)|$, where $\hat{F}_n$ is the empirical CDF.

For the discrete Poisson distribution we use **Pearson's chi-squared** test: $\chi^2 = \sum_k (O_k - E_k)^2 / E_k$, with bins merged to ensure $E_k \geq 5$.

3 Implementation

3.1 Simulation Engine

The engine uses `numpy.random.default_rng` (PCG64 algorithm), generating all 10^6 paths in a single vectorized call:

```
1 def simulate_normal(self, mu=0.0, sigma=1.0):
2     samples = self.rng.normal(loc=mu, scale=sigma,
3                               size=self.n_paths)
4     return self._build_result("Normal",
5                               {"mu": mu, "sigma": sigma},
6                               samples)
```

Listing 1: Vectorized Normal sampler

Skewness and kurtosis are computed via vectorized moment operations:

```
1 centred = samples - mean
2 skew    = (centred**3).mean() / (std**3 + 1e-12)
3 kurt    = (centred**4).mean() / (std**4 + 1e-12) - 3.0
```

Listing 2: Vectorized moment computation

3.2 Convergence Analyser

A single batch of 10^6 samples is drawn once, then sliced at 60 log-spaced checkpoints from $n = 100$ to $n = 10^6$. The convergence rate is estimated by OLS regression of $\log(SE_n)$ on $\log(n)$.

4 Results — Distribution Analysis

4.1 Empirical vs Theoretical Moments

Table 1: Empirical vs theoretical moments — $n = 10^6$ paths

Distribution	Emp. Mean	Th. Mean	Emp. Std	Th. Std	Emp. Skew	Th.
Normal ($\mu=0, \sigma=1$)	0.000098	0.000000	1.000483	1.000000	-0.001	0
Log-Normal ($\mu=0, \sigma=0.5$)	1.133568	1.133148	0.603999	0.603901	1.755	1
Exponential ($\lambda=2$)	0.499858	0.500000	0.498916	0.500000	1.997	2
Poisson ($\lambda=5$)	5.000647	5.000000	2.235591	2.236068	0.448	0

All empirical moments agree with theoretical values to at least 4 significant figures.

4.2 Goodness-of-Fit Tests

Table 2: Goodness-of-fit results — subsample of 10^4 paths

Distribution	Test	Statistic	p-value	Result
Normal	KS	0.010865	0.1871	Pass
Log-Normal	KS	0.007083	0.6946	Pass
Exponential	KS	0.008932	0.3999	Pass
Poisson	χ^2	16.421	0.2271	Pass

All p-values are well above $\alpha = 0.05$, confirming each sampler correctly implements the target distribution.

5 Convergence Analysis

5.1 Experimental Convergence Rates

Table 3: Empirical convergence rates vs CLT prediction of -0.50

Distribution	True Mean	Empirical Slope	Deviation
Normal ($\mu=0, \sigma=1$)	0.0000	-0.5091	$+0.0091$
Log-Normal ($\mu=0, \sigma=0.5$)	1.1331	-0.5031	$+0.0031$
Exponential ($\lambda=2$)	0.5000	-0.4867	-0.0133
Poisson ($\lambda=5$)	5.0000	-0.5066	$+0.0066$

All slopes are within 0.014 of the theoretical -0.50 . Small deviations reflect finite-sample variability in the OLS estimate.

5.2 Implications for Simulation Design

From equation (2), halving the standard error requires quadrupling the number of paths. This motivates variance reduction techniques: antithetic variates (using Z and $-Z$), control variates, and quasi-Monte Carlo sequences (Sobol, Halton) achieving $O((\log n)^d/n)$.

6 Application: European Call Option Pricing

6.1 Black-Scholes Framework

Under the risk-neutral measure, the stock price follows:

$$S_T = S_0 \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right], \quad Z \sim \mathcal{N}(0, 1)$$

The European call value is:

$$C = e^{-rT} \mathbb{E}^{\mathbb{Q}}[\max(S_T - K, 0)]$$

6.2 Results

Table 4: European call option pricing — $S_0=K=100$, $T=1\text{yr}$, $r=5\%$, $\sigma=20\%$

Method	Price	Relative Error
Black-Scholes analytical	10.4506	—
Monte Carlo ($n = 10^6$)	10.4284	0.21%
95% CI: (10.4035, 10.4533)		Std error: 0.01268

The Black-Scholes analytical price falls within the Monte Carlo 95% confidence interval, confirming the statistical validity of the estimate.

7 Conclusion

This project has demonstrated the full Monte Carlo pipeline from simulation to financial application. Key findings:

1. A vectorized Python engine simulates 10^6 paths per distribution in under one second.
2. Empirical moments match theory to 4+ significant figures; all goodness-of-fit tests pass.
3. CLT convergence rates are confirmed experimentally: slopes between -0.487 and -0.509 .
4. Monte Carlo option pricing achieves 0.21% relative error versus Black-Scholes at 10^6 paths.

Future work includes variance reduction techniques and extension to path-dependent pay-offs. A separate ongoing project applies neural networks to stochastic volatility model calibration.

References

- [1] Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. Springer.
- [2] Jäckel, P. (2002). *Monte Carlo Methods in Finance*. Wiley.
- [3] Black, F. & Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*, 81(3), 637–654.
- [4] Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [5] Virtanen, I. et al. (2020). SciPy 1.0. *Nature Methods*, 17, 261–272.